

Phát triển ứng dụng web

NodeJS
Part 2

Nội dung

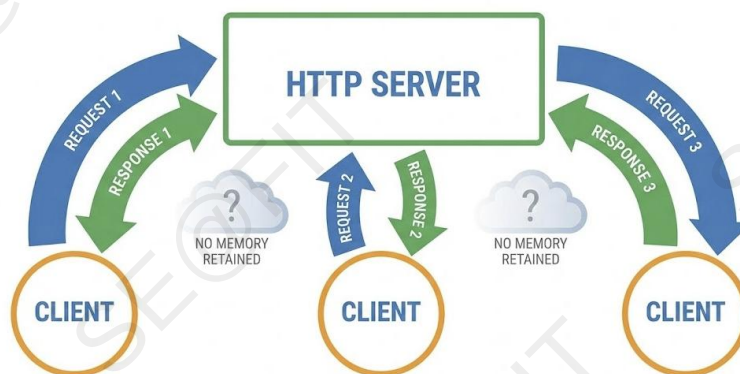
- ☐ Web server – Các vấn đề cơ bản
- ☐ RESTful API
- ☐ Database

Nội dung

- ❑ **Web server – Các vấn đề cơ bản**
 - ❑ **State Management**
 - ❑ Authentication, Authorization
 - ❑ Error Handling
 - ❑ RESTful API
 - ❑ Database

HTTP là giao thức stateless

- ❑ Mỗi request HTTP là **độc lập**:
 - ❑ Server không lưu giữ thông tin gì về client giữa các lần request khác nhau.



Khái niệm State Management

☐ **State:** thông tin cần được “nhớ” qua nhiều request:

- ☐ User đã đăng nhập hay chưa
- ☐ Vai trò: admin / user thường
- ☐ Giỏ hàng, ngôn ngữ, theme, ...

☐ **State Management:**

- ☐ Cách thiết kế, lưu trữ, và truy xuất state
- ☐ Có thể lưu ở:
 - ☐ **Server side:** Session, Database, cache...
 - ☐ **Client side:** Cookie, LocalStorage, SessionStorage...

Express.js trong quản lý state

☐ Express.js là framework phổ biến cho Node.js

- ☐ Không có “state” tích hợp sẵn
- ☐ Cung cấp **request**, **response** và middleware

☐ Quản lý state trong Express:

- ☐ Dùng session (middleware: express-session, ...)
- ☐ Dùng cookie (middleware: cookie-parser, ...)
- ☐ Kết hợp client-side như LocalStorage (qua JS ở phía front-end)

Cookie– Mẫu dữ liệu nhỏ trên trình duyệt

- ❑ **Cookie** là **object** (kích thước tối đa 4KB) dùng để lưu dữ liệu tạm thời cho ứng dụng web. Điểm khác biệt cơ bản là nơi lưu trữ **cookie** là ở trình duyệt.
- ❑ Cơ chế hoạt động:
 - ❑ Server gửi **Set-Cookie** header trong Response.
 - ❑ Trình duyệt lưu lại.
 - ❑ Trong **mọi** Request tiếp theo đến cùng domain đó, trình duyệt tự động đính kèm Cookie vào Header.
- ❑ Dữ liệu: Dạng chuỗi **key=value**.

```
res.cookie('visited', 'yes', { maxAge: 900000 });
```

```
> document.cookie  
← 'visited=yes'
```

Cookie - **HttpOnly**

- ❑ Vấn đề: Mặc định, JavaScript (`document.cookie`) có thể đọc và ghi đè Cookie.
- ❑ Nguy cơ bị đánh cắp Cookie để mạo danh nạn nhân (Session Hijacking).
- ❑ Giải pháp **HttpOnly**:
 - ❑ Khi có cờ này, Browser **CẤM** JavaScript truy cập Cookie đó.
 - ❑ Cookie vẫn được **tự động** gửi đi trong Request Header, nhưng code JS không truy xuất vào được.

Cookie - Secure và SameSite

☐ Flag **Secure**:

- ☐ Chỉ gửi Cookie nếu đang dùng giao thức **HTTPS**.
- ☐ Ngăn chặn nghe lén gói tin (Man-in-the-Middle) khi user dùng Wifi công cộng (HTTP thường).

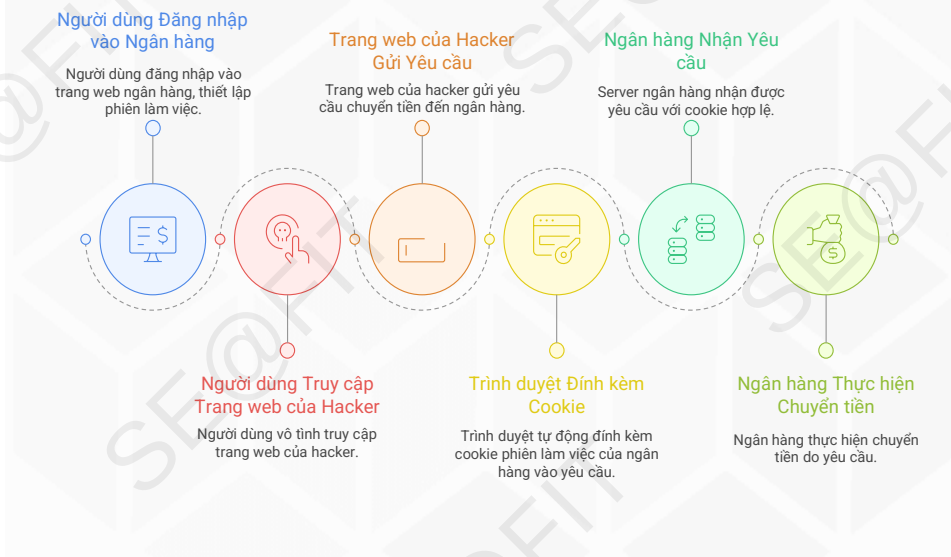
☐ Flag **SameSite** (Chống CSRF):

- ☐ Quy định khi nào được gửi Cookie nếu request đến từ một website khác (Cross-site).
- ☐ **SameSite=Strict**: Chỉ gửi cookie nếu đang ở chính trang web đó.
- ☐ **SameSite=Lax** (Mặc định hiện nay): Gửi cookie khi user điều hướng (bấm link) từ trang khác sang, nhưng **KHÔNG** gửi nếu trang khác gọi ngầm (nhúng ảnh, iframe, POST form). Cân bằng tốt.
- ☐ **SameSite=None**: Luôn gửi. Bắt buộc phải đi kèm Secure.

So sánh loại giá trị SameSite

Loại Request từ Site A -> Site B	Strict	Lax (Default)	None (+Secure)
Bấm Link ()	✗ Chặn	☑ Gửi	☑ Gửi
Gõ URL trực tiếp	✗ Chặn *	☑ Gửi	☑ Gửi
Form GET	✗ Chặn	☑ Gửi	☑ Gửi
Form POST	✗ Chặn	✗ CHẶN	☑ Gửi
Iframe (<iframe src="...">)	✗ Chặn	✗ CHẶN	☑ Gửi
Fetch / AJAX / Image	✗ Chặn	✗ CHẶN	☑ Gửi

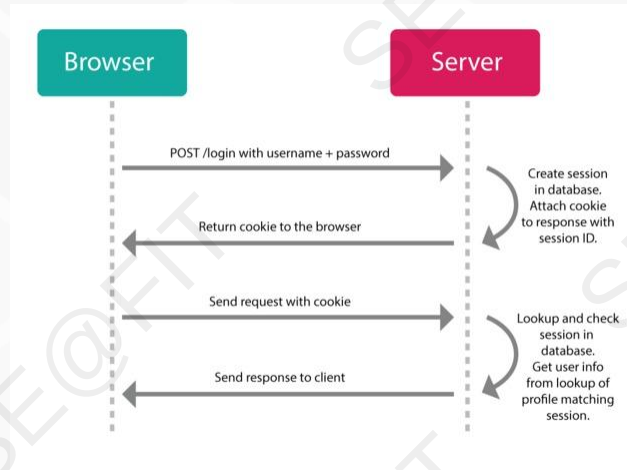
Ví dụ CSRF (Cross-Site Request Forgery)



Web Session

- ❑ **Web session** là **object** được sử dụng để lưu trữ dữ liệu tạm thời (RAM, File, Database) trong thời gian nhất định (thời gian người dùng tương tác với ứng dụng).
- ❑ Cấu trúc của **session** là **key-value** và được phân biệt bằng **session_id**.
- ❑ Một **session** bắt đầu khi **client** gửi **request** đến **server**, nó tồn tại xuyên suốt trong ứng dụng **web** và chỉ kết thúc khi hết thời gian **timeout** hoặc khi đóng ứng dụng. Giá trị của **session** sẽ được lưu trong một **file** trên **server**.

Quy trình triển khai session



Vấn đề Session Store (Memory vs. Database)

☐ Mặc định (MemoryStore):

- ☐ Vấn đề 1: Server restart -> Mất hết session (User bị logout).
- ☐ Vấn đề 2: Memory Leak (tràn RAM).
- ☐ Vấn đề 3: Không chạy được với nhiều Server (Load Balancing).
- ☐ **MemoryStore chỉ dùng cho development, không dùng cho production.**

☐ Database: lưu tất cả session vào **1 chỗ chung** (Redis/DB).

☐ Khi server nhận request:

- ☐ Lấy **sessionID** từ **cookie**.
- ☐ Truy cập Session Store (Redis/DB) lấy dữ liệu **session** tương ứng.
- ☐ Gắn vào **req.session**.

So sánh MemoryStore vs Database Store

Tiêu chí	MemoryStore (RAM Node)	Database / Redis / Mongo
Vị trí lưu trữ	RAM của từng process Node	Server riêng (Redis/DB), dùng chung nhiều instance
Mất dữ liệu khi restart	Có (mất hết session)	Thường không , tùy cấu hình
Hỗ trợ scale-out	Kém, mỗi instance giữ session riêng	Tốt, mọi instance truy cập cùng một Store
Độ phức tạp triển khai	Đơn giản, không cần service gì thêm	Phức tạp hơn, cần cài và cấu hình DB/Redis
Hiệu năng	Rất nhanh (truy cập RAM trực tiếp)	Nhanh nhưng vẫn qua mạng (Redis rất tốt)
Dùng cho môi trường	Dev / demo / project nhỏ	Production / hệ thống thực tế

LocalStorage là gì?

☐ Thuộc HTML5 Web Storage API

☐ LocalStorage:

- ☐ Lưu key-value trong trình duyệt vĩnh viễn (cho đến khi xóa thủ công).
- ☐ Dung lượng lớn (~5MB - 10MB).
- ☐ **Không gửi** kèm request HTTP

☐ SessionStorage:

- ☐ Giống LocalStorage nhưng mất dữ liệu khi đóng Tab/Window.

```
localStorage.setItem('key', 'value');  
console.log(localStorage.getItem('key'));  
sessionStorage.setItem('sessionKey', 'sessionValue');  
console.log(sessionStorage.getItem('sessionKey'));
```


Cách thức sử dụng LocalStorage

☐ Server:

- ☐ Dùng Session để xác thực & phân quyền

☐ Client:

- ☐ Dùng LocalStorage để lưu:
 - ☐ Cài đặt UI (theme, language)
 - ☐ Bộ nhớ “nháp” (draft form)
 - ☐ Kết quả API ít nhạy cảm (cache)

Session vs. Cookie vs. LocalStorage

Tiêu chí	Cookie	LocalStorage	Session (Server-side)
Nơi lưu dữ liệu	Client (Browser)	Client (Browser)	Server (RAM/DB)
Dung lượng	Nhỏ (4KB)	Lớn (5MB+)	Không giới hạn (tùy Server)
Tự động gửi lên Server	CÓ (trong Header)	KHÔNG	KHÔNG (Chỉ gửi SessionID qua Cookie)
Truy cập bởi JS	Chặn được (HttpOnly)	Luôn luôn	Client không thể truy cập trực tiếp
Mục đích chính	Session ID, Tracking	UI Settings, Cart (offline)	Thông tin User nhạy cảm, Auth state
Bảo mật	Chống XSS tốt (nếu HttpOnly)	Dễ dính XSS	An toàn nhất cho dữ liệu nhạy cảm

Nội dung

❑ Web server – Các vấn đề cơ bản

- ❑ *State Management*
- ❑ **Authentication, Authorization**
- ❑ Error Handling
- ❑ RESTful API
- ❑ Database

Quản lý người dùng hệ thống

Thay đổi thông tin dữ liệu

Việc thay đổi thông tin dữ liệu hợp lý và nhất quán có ý nghĩa rất quan trọng.



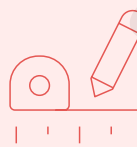
Xác định quyền người dùng

Việc xác định người dùng nào được phép làm những gì đối với hệ thống là bắt buộc.



Xây dựng kiến trúc lưu vết

Để dễ dàng kiểm soát hệ thống thì việc xây dựng kiến trúc giúp lưu vết hoạt động cũng là yêu cầu cần thiết và bắt buộc đặc biệt với hệ thống lớn.



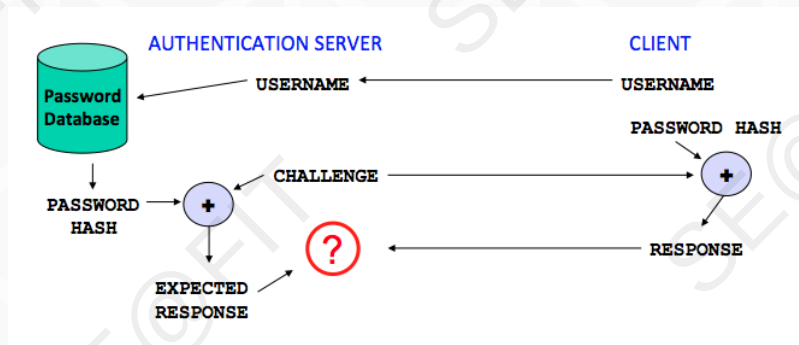
Authentication

- ☐ **Authentication** là xác thực, là quá trình kiểm tra danh tính của người dùng hoặc một hệ thống khác đến hệ thống hiện tại thông qua một hệ thống xác thực.
- ☐ Đây là bước ban đầu của mọi hệ thống có yếu tố người dùng. Nếu không có bước xác thực này, hệ thống sẽ không biết được người đang truy cập vào hệ thống là ai để có các phản hồi phù hợp.
- ☐ Một số phương thức xác thực thông dụng
 - ☐ Mật khẩu
 - ☐ Khóa
 - ☐ Sinh trắc học

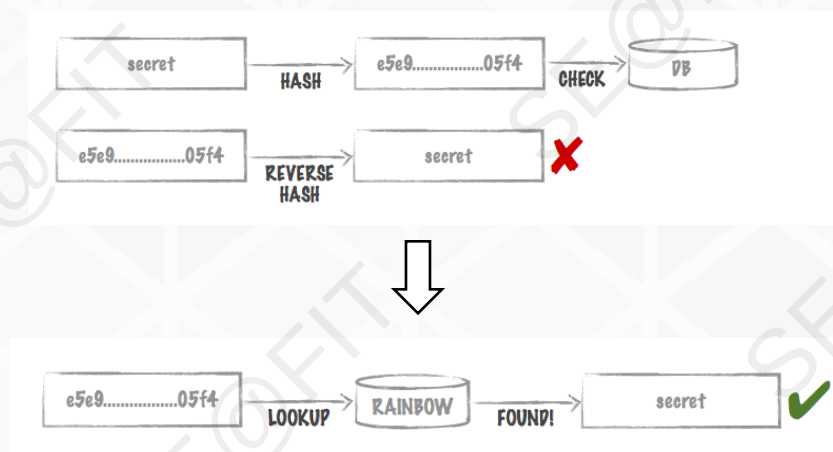
Xác thực bằng mật khẩu (Password & PIN)

- ☐ Mật khẩu là một trong những phương pháp đơn giản và dễ triển khai nhất. Thường mỗi hệ thống sẽ lưu lại mật khẩu ở dạng đã được mã hóa một chiều (**md5, sha1, ...**) để đảm bảo mật khẩu có bị lộ cũng không thể khôi phục thành chuỗi gốc.
- ☐ Phương pháp này còn có nhiều biến thể như thiết kế dạng **Swipe Pattern PIN** (trong các điện thoại android) hoặc mật khẩu dùng một lần (dùng cho các chức năng quan trọng).

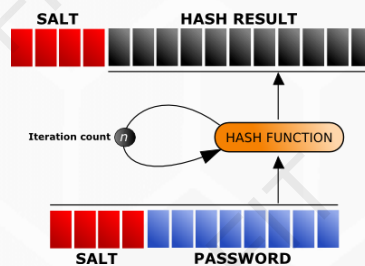
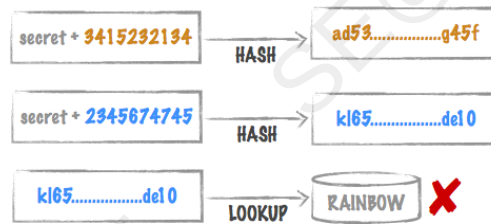
Mô hình cơ bản



Vấn đề phá mã



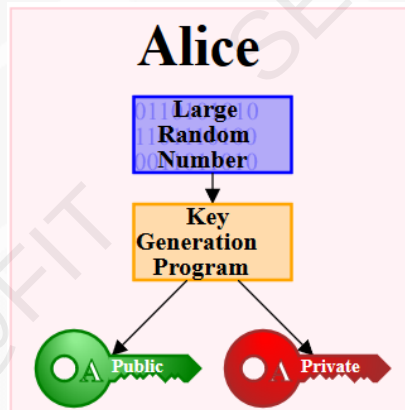
Giải pháp thông dụng



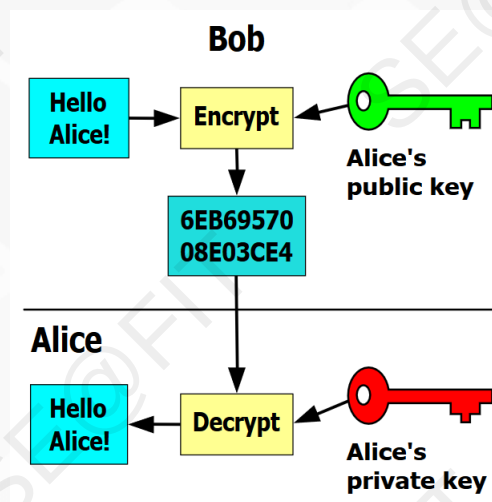
Xác thực bằng khóa công khai

- ❑ Phương pháp này dựa trên thuật toán mã hóa khóa công cộng (**public key**) và khóa cá nhân (**private key**). Phương pháp này giúp cho người đăng nhập không cần nhớ thông tin gì về đăng nhập như phương pháp mật khẩu. Để đăng nhập vào hệ thống người dùng chỉ cần có khóa cá nhân (**private key**) trên máy và đăng nhập vào hệ thống (nếu đã khai báo với khóa công cộng của người dùng). Cách này thường được áp dụng và bật với các hệ thống quản trị **server**.

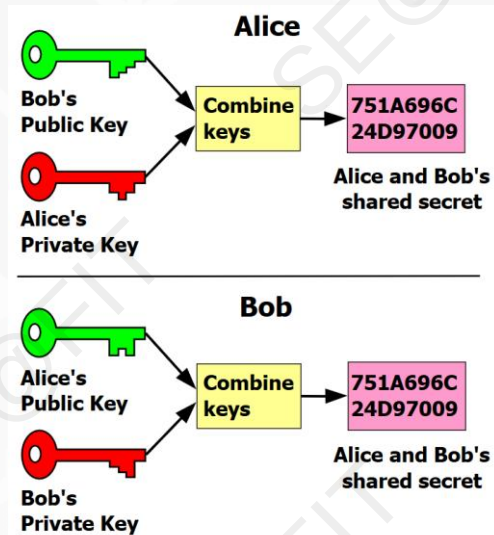
Tạo khóa



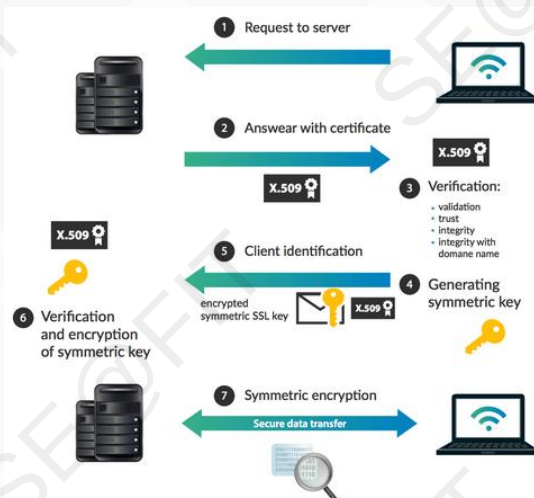
Áp dụng



Kết hợp



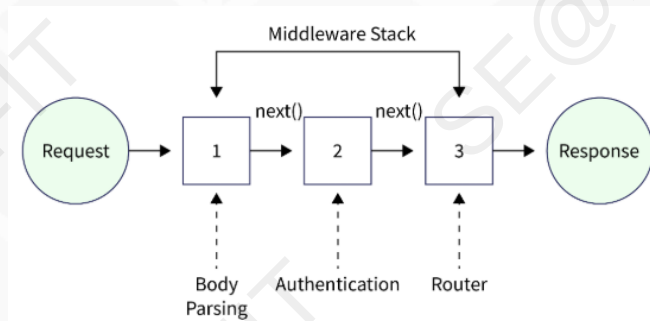
SSL



Authorization

- ❑ **Authorization** là quá trình xác định xem một người dùng có quyền truy cập một tài nguyên cụ thể hoặc để thực hiện một số hành động hay không.
- ❑ **Authorization** thường được quản lý dựa trên **vai trò (role)**, **quyền hạn (permission)** hoặc **chính sách (policy)** được cấu hình để kiểm soát mức độ truy cập được cấp cho các đối tượng khác nhau.
- ❑ Các hình thức phân quyền thường gặp là:
 - ❑ **Role-based authorization**
 - ❑ **Object-based authorization**

Triển khai trong express



```
export const requireAdmin = (req, res, next) => {  
  if (req.user.role === 'admin') {  
    next();  
  } else {  
    res.status(403).json({ message: 'Access denied, admin only' });  
  }  
};
```


Nội dung

- ❑ **Web server – Các vấn đề cơ bản**

- ❑ *State Management*
 - ❑ *Authentication, Authorization*
 - ❑ **Error Handling**
- ❑ RESTful API
- ❑ Database

Error Handling

- ❑ **Error handling** là một khía cạnh quan trọng của lập trình, bao gồm việc quản lý và phản ứng với các lỗi hoặc ngoại lệ có thể xảy ra trong quá trình thực thi của một chương trình.
- ❑ Thực hiện **error handling** đúng cách giúp đảm bảo rằng một chương trình có thể xử lý một cách mềm dẻo các tình huống bất ngờ và cung cấp phản hồi có ý nghĩa cho người dùng hoặc nhà phát triển.

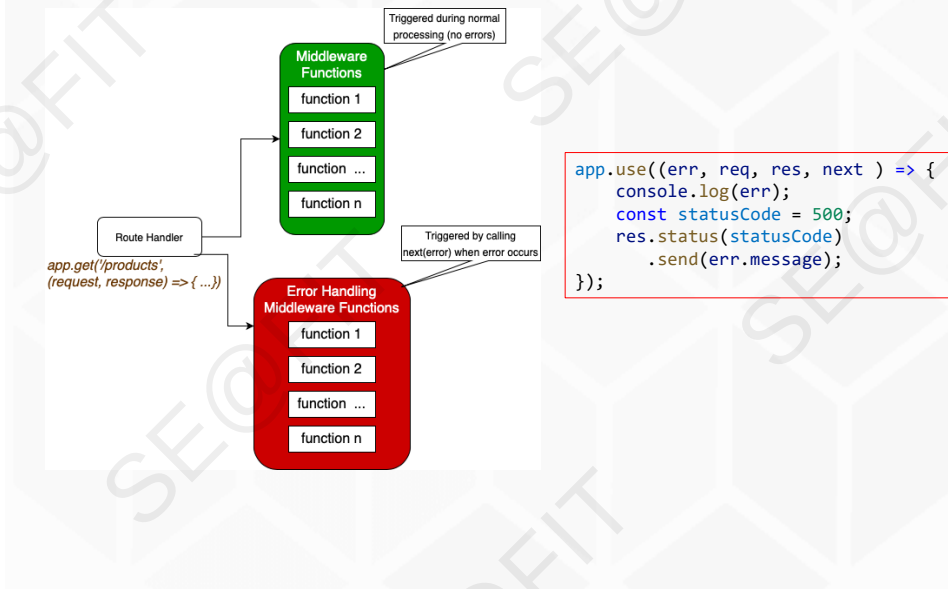
Error Handling (tt.)

- ❑ Các loại lỗi thường gặp trong ứng dụng
 - ❑ Operational errors: lỗi xảy ra trong quá trình hoạt động của ứng dụng, phần lớn là lỗi thao tác, nhập liệu, đầy dữ liệu,...
 - ❑ Programmer errors: lỗi xảy ra do những đoạn mã không mong muốn, không được kiểm soát tốt như truy suất đối tượng không tồn tại (undefined),...
- ❑ Quan tâm và xử lý các lỗi như thế nào cho hợp lý là điều quan trọng để phát triển 1 ứng dụng tốt.
- ❑ Có nhiều phương pháp để xử lý lỗi, và cách tiếp cận cụ thể có thể thay đổi tùy thuộc vào ngôn ngữ lập trình và bản chất của ứng dụng.

Express.js – Error Handling

- ❑ **Express** đi kèm với một trình xử lý lỗi tích hợp sẵn, đảm nhiệm việc xử lý bất kỳ lỗi nào có thể gặp phải trong ứng dụng. **Middleware error-handling** mặc định này được thêm vào cuối ngăn xếp các **middleware**.
- ❑ Các lỗi xảy ra trong mã **đồng bộ** bên trong các trình xử lý **route** và **middleware** không cần thêm bất kỳ xử lý đặc biệt nào. Nếu mã **đồng bộ** ném ra một **lỗi**, thì **Express** sẽ bắt và xử lý nó.
- ❑ Đối với các **lỗi** được trả về từ các hàm **bất đồng bộ** được gọi bởi các trình xử lý **route** và **middleware** thì phải truyền chúng đến hàm **next()**, nơi **Express** sẽ bắt và xử lý chúng.

Sử dụng Middleware



Ví dụ

```
app.get('/', (req, res) => {  
  const uobj = null;  
  console.log(uobj.test);  
  res.send('error');  
});
```

localhost:3000

TypeError: Cannot read properties of null (reading 'test')

```
app.get('/', async (req, res) => {  
  const uobj = null;  
  console.log(uobj.test);  
  res.send('error');  
});
```

This site can't be reached

localhost refused to connect.

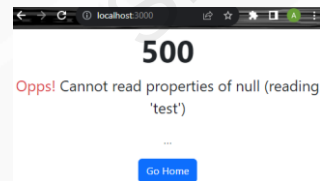
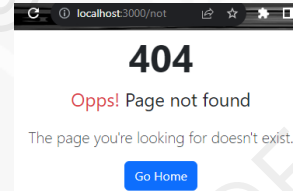
```
app.get('/', async (req, res, next) => {  
  try {  
    const uobj = null;  
    console.log(uobj.test);  
    res.send('error');  
  } catch (error) {  
    next(error);  
  }  
});
```

localhost:3000

TypeError: Cannot read properties of undefined (reading 'test')
at Timeout._onTimeout (3000ms) (server.js:33:29)

Custom error handlers

```
/// End position in the code
app.use((req, res, next) => {
  res.status(404).render('error', {
    title: 'not found',
    statusCode: 404,
    msg: 'Page not found',
    description: "The page you're looking
for doesn't exist."
  });
});
app.use(function (err, req, res, next) {
  const statusCode = err.statusCode ?
err.statusCode : 500;
  res.status(statusCode).render('error', {
    title: 'error page',
    statusCode: statusCode,
    msg: err.message,
    description: '...'
  });
});
});
```



Sử dụng Custom Error Classes

```
class MyError extends Error {
  constructor(statusCode, message, desc = '') {
    super(message);
    this.statusCode = statusCode;
    this.desc = desc;
    // Capture the stack trace
    Error.captureStackTrace(this, this.constructor);
  }
}
```



```
app.get('/', (req, res, next) => {
  try {
    const data = fs.readFile('./NoSuchFileExists', 'utf-8');
  } catch (error) {
    return next(new MyError(500, 'Error reading file', error.message));
  }
  //...
});
```

Async Error Handling

- ❑ Thực hiện error-handling trong asynchronous code (**async/await**) trong **Express.js** yêu cầu được quan tâm đặc biệt bởi vì hàm **async không tự động truyền error đến middleware error-handling**.

```
app.get('/', async (req, res, next) => {  
  try {  
    const data = await fs.readFile('./NoSuchFileExists', 'utf-8');  
  } catch (error) {  
    return next(new MyError(500, 'Error reading file', error.message));  
  }  
  //...  
});
```

- ❑ Sử dụng Wrapper function

```
function asyncWrapper(fn) {  
  return function (req, res, next) {  
    Promise.resolve(fn(req, res, next)).catch(next);  
  };  
}
```

Common HTTP status codes

- ❑ 1xx (Informational): Informational - The request was received, continuing process.
- ❑ 2xx (Success): Success - The request was successfully received, understood, and accepted.
 - ❑ 200 OK: The request was successful.
 - ❑ 201 Created: The request was successful, and a new resource was created.
 - ❑ 204 No Content: The request was successful, but there is no data to return.
- ❑ 3xx (Redirection): Redirection - Further action needs to be taken to complete the request.
 - ❑ 301 Moved Permanently: The resource has been moved permanently to a new location.
 - ❑ 302 Found (or Moved Temporarily): The resource has been temporarily moved to a new location.

Common HTTP status codes (cont.)

- ❑ 4xx (Client Error): Client Error - The request contains bad syntax or cannot be fulfilled.
 - ❑ 400 Bad Request: The request is invalid or cannot be understood.
 - ❑ 403 Forbidden: The server refused to fulfill the request.
 - ❑ 404 Not Found: The requested resource could not be found.
- ❑ 5xx (Server Error): Server Error - The server failed to fulfill a valid request.
 - ❑ 500 Internal Server Error: A generic error message indicating an internal server error.
 - ❑ 503 Service Unavailable: The server is not ready to handle the request. Common causes include temporary overloading or maintenance of the server.

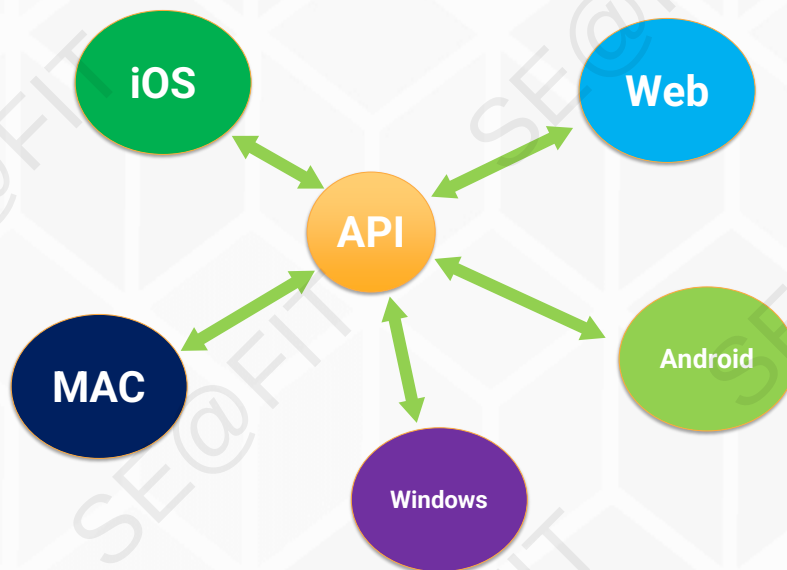
Nội dung

- ❑ *Web server – Các vấn đề cơ bản*
- ❑ **RESTful API**
- ❑ Database

RESTful API là gì?

- ❑ **REST** (**RE**presentational **State** **T**ransfer): Một phong cách kiến trúc cho các dịch vụ web, tập trung vào việc sử dụng HTTP để trao đổi dữ liệu giữa Client và Server.
- ❑ Tập trung vào **resource** và biểu diễn (representation) như JSON.
- ❑ “**RESTful**” = API tuân thủ các nguyên tắc REST ở mức hợp lý

Benefits of Using APIs



REST – Các nguyên tắc

- ☐ **Client – Server**: tách biệt trách nhiệm
- ☐ **Stateless**: Mỗi request phải **tự mang đủ thông tin** để server xử lý.
- ☐ **Cacheable**: Response nên nói rõ **có cache được không** và cache thế nào.
- ☐ **Uniform Interface**: thống nhất về URI, state, method,...
- ☐ **Layered System**: xây dựng server đa tầng (API Gateway, Load balancer, Reverse proxy (Nginx), Cache layer, Auth service...)
- ☐ **Code-on-Demand**: (tùy chọn, hiếm dùng) Server gửi code thực thi về client (ví dụ JS) để mở rộng chức năng.

Resource - based

- ☐ **Resource** = danh từ (noun): users, orders, products
- ☐ Một **resource** gồm:
 - ☐ **Identifier** (URI)
 - ☐ **Representation** (JSON)
 - ☐ **State** (trạng thái dữ liệu)
- ☐ **URI Design**: nguyên tắc đặt đường dẫn
 - ☐ Dùng danh từ số nhiều: **/users**, **/products**
 - ☐ Tài nguyên con: **/users/:id/orders**
 - ☐ Tránh động từ trong URI: **/getUser** ↔ **/users/:id**
 - ☐ Tránh lộ chi tiết nội bộ: **/db/users**

Common HTTP Methods

- ❑ **GET:** Lấy thông tin tài nguyên từ server.
- ❑ **POST:** Tạo mới một tài nguyên trên server.
- ❑ **PUT/PATCH:** Cập nhật một tài nguyên hiện có trên server.
- ❑ **DELETE:** Xóa một tài nguyên trên server.

Các thành phần của RESTful API

- ❑ **Endpoint:** URL đại diện cho tài nguyên.
- ❑ **Method:** Phương thức HTTP (GET, POST, PUT/PATCH, DELETE).
- ❑ **Headers:** Thông tin bổ sung gửi kèm theo **request** hay **response** (ví dụ: Content-Type, Authorization).
- ❑ **Body:** Dữ liệu gửi kèm theo **request** (đối với POST và PUT).

HTTP Status Codes quan trọng

- ☐ **200 OK:** Thành công.
- ☐ **201 Created:** Tạo mới thành công.
- ☐ **204 No Content:** Xóa thành công (không body).
- ☐ **400 Bad Request:** Lỗi do Client gửi dữ liệu sai.
- ☐ **401 Unauthorized:** Chưa đăng nhập.
- ☐ **403 Forbidden:** không đủ quyền
- ☐ **404 Not Found:** sai địa chỉ (không tồn tại).
- ☐ **500 Internal Server Error:** Server lỗi (không leak info).

Validation

- ☐ Validation:
 - ☐ Ngăn dữ liệu bẩn vào hệ thống
 - ☐ Trả lỗi rõ ràng cho client
 - ☐ Giảm bug và giảm “if-else” trong service
- ☐ Triển khai theo middleware
 - ☐ Validate **params / query / body**
 - ☐ Chuẩn hóa dữ liệu (trim, default, type coercion nếu có)
 - ☐ Nếu sai → trả lỗi **400 hoặc 422** theo format thống nhất
 - ☐ (**422 Unprocessable Entity:** request **không đạt rule validation**)
- ☐ Package hỗ trợ: **Zod, Joi,...**

Error Handling chuẩn Express

- ☐ Không **try/catch** rải rác thiếu kiểm soát.
- ☐ Sử dụng:
 - ☐ **next(err)** để đẩy về error middleware
 - ☐ Custom error class: `AppError(code, status, message)`
- ☐ Xử lý lỗi từ bất đồng bộ
 - ☐ Luôn dùng Async handler wrapper cho mọi controller async

Logging

- ☐ Mục tiêu logging trong REST API:
 - ☐ Debug nhanh lỗi production
 - ☐ Trace theo từng request (end-to-end)
 - ☐ Đo hiệu năng: latency, error rate
 - ☐ Hỗ trợ audit (ai làm gì, khi nào)
- ☐ Triển khai:
 - ☐ Gắn **requestId** cho mọi request
 - ☐ Log có cấu trúc (structured) + đủ ngữ cảnh
 - ☐ Không log dữ liệu nhạy cảm (password, token...)

Caching cơ bản

☐ Mục tiêu cache cho **REST API** (chủ yếu với **GET**):

- ☐ Giảm tải server/DB
- ☐ Tăng tốc phản hồi cho client
- ☐ Tăng độ ổn định khi traffic tăng

☐ **HTTP headers** quan trọng:

- ☐ **Cache-Control**
- ☐ **ETag + If-None-Match**
- ☐ **Last-Modified + If-Modified-Since**

Idempotency cho POST

☐ Vấn đề: **Client retry POST** (timeout/mạng chậm chèn) → **tạo trùng resource** (đặc biệt đơn hàng/thanh toán).

☐ Giải pháp: **Header Idempotency-Key**

- ☐ **Client** gửi **Idempotency-Key**: <unique-key>
- ☐ **Server** lưu kết quả theo **key** trong một khoảng thời gian
- ☐ **Retry** cùng **key** → trả lại kết quả cũ thay vì tạo mới

Vấn đề bảo mật cho REST API

- ☐ Injection (SQL/NoSQL)
- ☐ Broken authn/authz (JWT sai, lộ secret, XSS, không check authz)
- ☐ Rate abuse (bruteforce, thiếu pagination)
- ☐ CORS misconfig (thiếu cấu hình)
- ☐ Sensitive data exposure
- ☐ Mass assignment (client set field không cho phép)

Security headers & CORS

- ☐ **Security headers** là các HTTP headers giúp trình duyệt/clients xử lý an toàn hơn: giảm rủi ro XSS, clickjacking, MIME sniffing...
 - ☐ Sử dụng helmet()
- ☐ **CORS (Cross-Origin Resource Sharing)** là cơ chế **trình duyệt** dùng để chặn/cho phép JavaScript từ origin A gọi tài nguyên ở origin B. Cấu hình CORS để:
 - ☐ Chỉ cho phép **origin** hợp lệ
 - ☐ Chỉ cho phép **methods/headers** cần thiết
 - ☐ Quyết định có dùng **credentials** (cookie) không

JWT

- ❑ **JSON Web Token (JWT)** là một tiêu chuẩn mở (**RFC 7519**) định nghĩa một cách gọn nhẹ và tự chứa thông tin an toàn để truyền tải giữa các bên dưới dạng một đối tượng **JSON**.
- ❑ Thông tin này có thể được xác minh và tin tưởng vì nó đã được ký số (digitally signed). **JWT** có thể được ký bằng cách sử dụng một khóa bí mật (với thuật toán **HMAC**) hoặc 1 cặp khóa public/private sử dụng **RSA** hoặc **ECDSA**.

Cấu trúc JWT

- ❑ Token là 1 chuỗi gồm 3 phần: **header**, **payload** và **signature**. Những phần này được phân tách bởi dấu **'.'**.
 - ❑ **Header**: Thường gồm hai phần: loại mã thông báo, thường là JWT, và thuật toán ký, như HMAC, SHA256 hoặc RSA.
 - ❑ **Payload**: Chứa các quyền. Quyền là các tuyên bố về một thực thể (thường là người dùng) và dữ liệu bổ sung.
 - ❑ **Signature**: Để tạo ra phần chữ ký, lấy tiêu đề đã mã hóa, dữ liệu chuyển tải đã mã hóa, một khóa bí mật, thuật toán được chỉ định trong header và ký nó.

API Documentation

- ☐ API là hợp đồng giữa các hệ thống FE/BE/mobile
- ☐ Docs giúp:
 - ☐ Dùng đúng endpoint, đúng payload, đúng status code
 - ☐ Giảm hỏi–đáp, giảm hiểu nhầm, giảm bug tích hợp
 - ☐ Onboard nhanh người mới
 - ☐ Dễ test, dễ mock, dễ maintain
- ☐ OpenAPI/Swagger là tiêu chuẩn phổ biến.

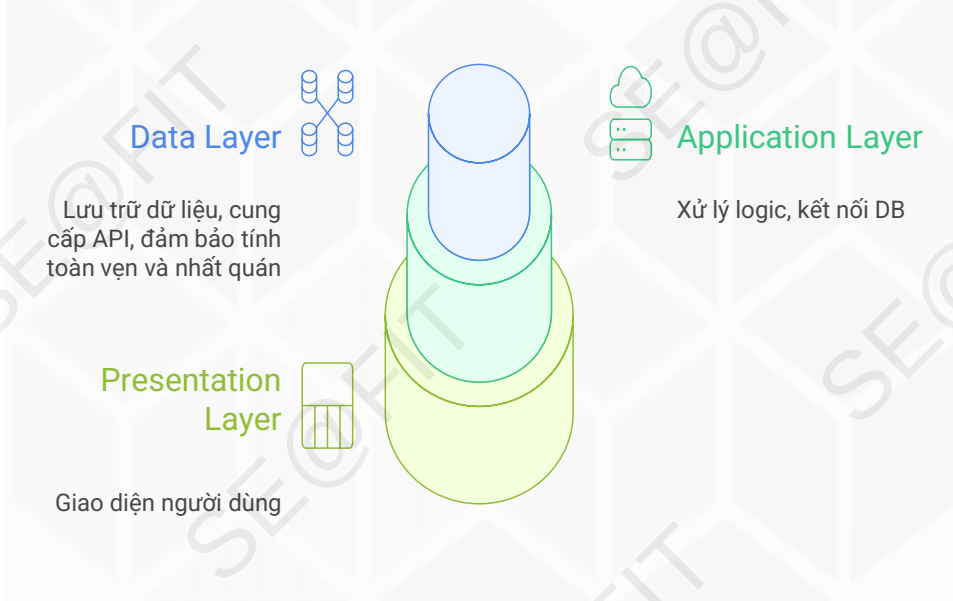
Nội dung

- ☐ *Web server – Các vấn đề cơ bản*
- ☐ *RESTful API*
- ☐ **Database**
 - ☐ **Cơ sở dữ liệu cho ứng dụng web**
 - ☐ Kết nối cơ sở dữ liệu trong node.js
 - ☐ Knex.js

Vai trò của Cơ sở dữ liệu

- ❑ Cơ sở dữ liệu là thành phần cốt lõi của mọi ứng dụng web.
- ❑ Vấn đề của Ứng dụng "Vô danh" (**Stateless**):
 - ❑ Dữ liệu lưu trên RAM (ví dụ: `let users = [];`).
 - ❑ Dữ liệu biến mất khi máy chủ khởi động lại.
 - ❑ Không thể chia sẻ dữ liệu giữa nhiều phiên bản máy chủ (**scaling**).
 - ❑ Khó khăn trong việc truy vấn và quản lý dữ liệu phức tạp.
- ❑ Giải pháp: Cơ sở dữ liệu (**Database**)
 - ❑ Lưu trữ dữ liệu bền vững (**Persistent Storage**) trên đĩa cứng.
 - ❑ Cung cấp cơ chế truy vấn (**Query**) mạnh mẽ.
 - ❑ Đảm bảo toàn vẹn dữ liệu (**ACID**).
 - ❑ Quản lý truy cập đồng thời (**Concurrency**).

Vị trí của CSDL trong kiến trúc ứng dụng web



Phân loại cơ sở dữ liệu

Loại	Đặc điểm	Ví dụ
Quan hệ (Relational)	Dữ liệu có cấu trúc, bảng, cột, khóa	PostgreSQL, MySQL, SQLite
Phi quan hệ (NoSQL)	Linh hoạt, không ràng buộc schema, JSON document	MongoDB, Firebase, Redis
NewSQL / Cloud DB	Hiệu năng cao, mở rộng dễ	CockroachDB, Supabase, PlanetScale

Mô hình dữ liệu và thiết kế CSDL

- ❑ **Thực thể (Entity)** → ví dụ: *User, Product, Order*.
- ❑ **Thuộc tính (Attributes)** → ví dụ: *name, email, price*.
- ❑ **Quan hệ (Relationships)** → 1-1, 1-n, n-n.
- ❑ **Khóa (Keys):**
 - ❑ Khóa chính (Primary key).
 - ❑ Khóa ngoại (Foreign key).
- ❑ **Chuẩn hóa dữ liệu (Normalization).**
- ❑ **Sơ đồ minh họa:**
 - ERD: **User (1) – (n) Order (n) – (1) Product**

Các hệ quản trị cơ sở dữ liệu (DBMS) phổ biến

- ☐ **PostgreSQL** – mã nguồn mở, mạnh mẽ, hỗ trợ JSON.
- ☐ **MySQL** – phổ biến, dễ dùng, nhiều tài liệu.
- ☐ **SQLite** – nhẹ, không cần server.
- ☐ **MongoDB** – NoSQL document, lưu dạng JSON.
- ☐ **Supabase** – nền tảng DB cloud dựa trên PostgreSQL.

Migrations

- ☐ **Migrations:** Quản lý thay đổi cấu trúc DB theo thời gian.
 - ☐ Tạo bảng (create tables)
 - ☐ Sửa đổi cấu trúc (alter tables)
 - ☐ Xóa bảng (drop tables)
- ☐ Có thể “di chuyển tiến” (*migrate up*) hoặc “quay lại” (*rollback*).
- ☐ Lợi ích:
 - ☐ Quản lý lịch sử thay đổi CSDL.
 - ☐ Dễ dàng rollback khi có lỗi.
 - ☐ Triển khai đồng bộ giữa nhiều môi trường.

Seeds

- ❑ **Seeds** là các tệp dùng để **chèn dữ liệu mẫu hoặc khởi tạo**.
- ❑ Thường dùng cho:
 - ❑ Dữ liệu mặc định (roles, categories, admin account).
 - ❑ Dữ liệu giả lập cho phát triển, demo, test.
- ❑ Lợi ích:
 - ❑ Tạo dữ liệu mẫu nhanh chóng.
 - ❑ Dễ test API hoặc giao diện.
 - ❑ Giúp CI/CD hoặc staging environment có dữ liệu nhất quán.

Hiệu năng và tối ưu hóa cơ sở dữ liệu

- ❑ Đối với ứng dụng web thì việc tối ưu hóa DB là nhiệm vụ thường xuyên của backend developer.

Đánh giá hiệu năng

Đánh giá hiệu năng bằng cách sử dụng EXPLAIN / ANALYZE.



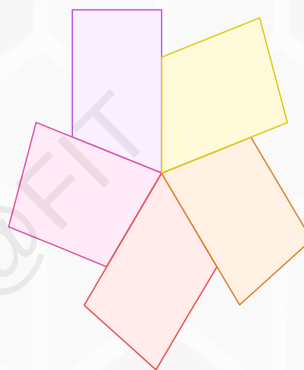
Sử dụng connection pool

Sử dụng connection pool để quản lý kết nối cơ sở dữ liệu.



Dùng cache

Sử dụng cache như Redis hoặc Memory Store để lưu trữ dữ liệu.



Tối ưu truy vấn

Tối ưu truy vấn bằng cách sử dụng index, limit và pagination.



Giảm JOIN phức tạp

Giảm các JOIN phức tạp để cải thiện hiệu suất.

Bảo mật cơ sở dữ liệu



Mã hóa mật khẩu

Sử dụng các thuật toán mạnh mẽ để bảo vệ mật khẩu.



Quản lý chuỗi kết nối

Lưu trữ chuỗi kết nối trong các biến môi trường để bảo mật.



Kiểm soát truy cập

Hạn chế quyền truy cập dựa trên vai trò để ngăn chặn truy cập trái phép.



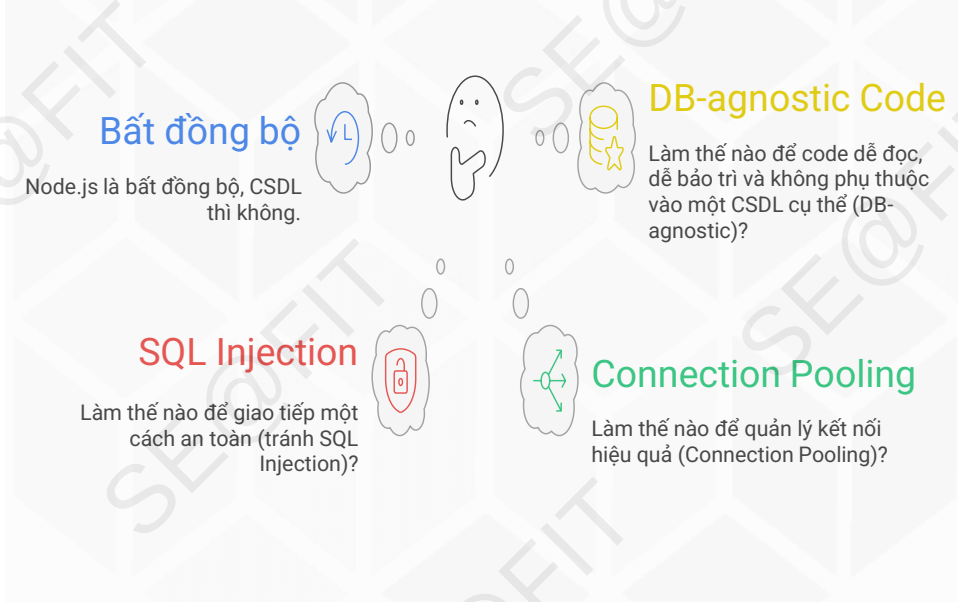
Sao lưu và khôi phục

Thực hiện sao lưu định kỳ để đảm bảo phục hồi dữ liệu.

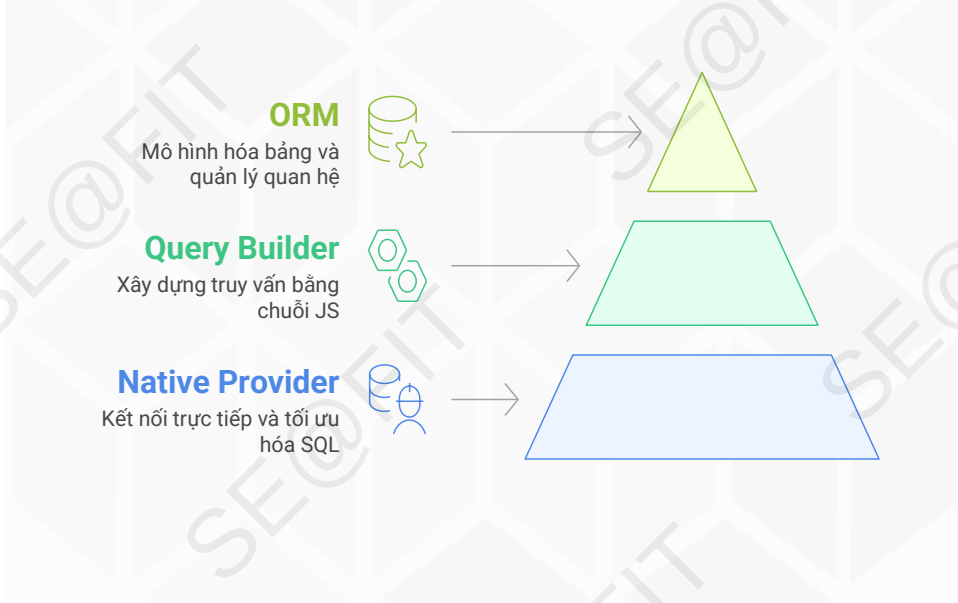
Nội dung

- ☐ Web server – Các vấn đề cơ bản
- ☐ RESTful API
- ☐ **Database**
 - ☐ Cơ sở dữ liệu cho ứng dụng web
 - ☐ **Kết nối cơ sở dữ liệu trong node.js**
 - ☐ Knex.js

Node.js và CSDL



Ba Cấp độ Trừu tượng



So sánh

Tiêu chí	pg-promise (Native)	Knex (QB)	Sequelize (ORM)
Độ trừu tượng	Thấp	Vừa	Cao
Tốc độ phát triển	TB	Nhanh	Rất nhanh (với CRUD)
Kiểm soát SQL	Tối đa	Cao (có thể raw)	Thấp-Vừa (raw khi cần)
Migrations/Seeds	Thủ công	Built-in	Có (CLI)
Học/Onboarding	Dễ nếu biết SQL	Dễ vừa	Dễ cho CRUD, phức tạp với associations
Hiệu năng	Tốt	Tốt	Tốt-TB (tùy sử dụng)
Khả năng đa DB	Postgres focus	Nhiều DB	Nhiều DB

Native Provider (pg-promise)

- ❑ **pg-promise** là một thư viện "bao bọc" (wrapper) nâng cao cho node-postgres (thư viện pg gốc).
- ❑ **pg-promise** bổ sung các tính năng cực kỳ quan trọng:
 - ❑ Tự động quản lý Connection Pooling.
 - ❑ Hỗ trợ **Promise** (thay vì callback hell).
 - ❑ Quản lý **Transaction** mạnh mẽ.
 - ❑ Định dạng (formatting) truy vấn mạnh mẽ.

Query Builder (Knex.js)

- ❑ **Knex.js** là một "**SQL Query Builder**" đa nền tảng (hỗ trợ Postgres, MySQL, SQLite, MS SQL...).
- ❑ **Tư tưởng:** Thay vì viết chuỗi SQL thì sử dụng các hàm JS để *xây dựng* truy vấn. Knex sẽ dịch code JS thành câu SQL phù hợp cho CSDL đang dùng.
- ❑ **Điểm mạnh:**
 - ❑ Database-Agnostic
 - ❑ Migrations
 - ❑ Dễ đọc
 - ❑ Tự động chống SQL Injection

ORM (Sequelize)

- ❑ Sequelize là một ORM (Object-Relational Mapping) "phổ biến" cho Node.js. Hỗ trợ Postgres, MySQL, MariaDB, SQLite và MS SQL Server.
- ❑ **Ý tưởng:** Ẩn hoàn toàn SQL khỏi developer.
- ❑ **Điểm mạnh:**
 - ❑ Phát triển nhanh
 - ❑ Trừu tượng hóa
 - ❑ Validations
 - ❑ Database-Agnostic

Lựa chọn

Native (pg-promise)

Chuyên SQL, cần kiểm soát tuyệt đối, ứng dụng nhỏ/gọn, vi dịch vụ chuyên biệt.



Query Builder (Knex)

Muốn migrations/seeds tốt, viết query JS tiện, vẫn có đường lui raw, đa DB.



ORM (Sequelize)

CRUD nhiều, team đông, muốn model hoá, validations, hooks, associations – chấp nhận học cách tối ưu ORM.



Nội dung

- ☐ Web server – Các vấn đề cơ bản
- ☐ RESTful API
- ☐ **Database**
 - ☐ Cơ sở dữ liệu cho ứng dụng web
 - ☐ Kết nối cơ sở dữ liệu trong node.js
 - ☐ **Knex.js**

Knex.js – SQL Query Builder mạnh mẽ cho Node.js

- ❑ **Knex** là Query Builder hỗ trợ nhiều hệ quản trị: PostgreSQL, MySQL, SQLite, MSSQL, Oracle...
- ❑ Tạo **query** bằng JavaScript: an toàn hơn nối chuỗi SQL → tự động **bind params**.
- ❑ Tích hợp dễ dàng với Express.js trong mô hình MVC.
- ❑ Cung cấp công cụ migrations, seeds, transactions, connection pooling...

Cài đặt

- ❑ Cài đặt Knex và driver CSDL
`npm install knex pg`
- ❑ Tạo file knexfile.js để định nghĩa các môi trường:
`npx knex init`
- ❑ File **knexfile.js**
 - ❑ Chứa cấu hình db client, connection, pool, migrations, seeds,...
- ❑ Tạo file migrations (đánh số tự động) và seed
`npx knex migrate:make <name>`
`npx knex seed:make <filename>`
- ❑ Lệnh chạy migrate, seed
`knex migrate:latest[/rollback]`
`knex seed:run`

Chuỗi (chaining) và Nguyên tắc

- ❑ Mỗi **query** là một chuỗi phương thức, kết thúc bằng **await** để thực thi.
- ❑ **Không concatenate string** thủ công; luôn dùng **where** với **placeholder**.
- ❑ Kiểm soát cột trả về (đừng lạm dụng **select('*')**)
- ❑ Luôn **bắt lỗi**: **try/catch**, **log**, chuẩn hoá **error** (Boom, http-errors).

Truy vấn: Query core

- ❑ Chọn bảng & cột

```
await db('users').select('id', 'email', 'created_at');
await db.select('id', 'name').from('products');
```

- ❑ Điều kiện where

```
// Lấy user có id = 1
await db('users').where({ id: 1 });
// Lấy user có email chứa '@example.com'
await db('users').where('email', 'ilike', '%@example.com');
// Lấy user có status là 'active' hoặc 'pending'
await db('users').whereIn('status', ['active', 'pending']);
// Lấy user có age nằm trong khoảng [18, 30]
await db('users').whereBetween('age', [18, 30]);
// Lấy user có role là 'admin' hoặc 'manager'
await db('users').where(builder => {
  builder.where('role', 'admin').orWhere('role', 'manager')
});
```

Truy vấn: Query core (tt)

❑ Sắp xếp, giới hạn, phân trang

```
// Lấy 10 sản phẩm đắt nhất
await db('products').orderBy('price', 'desc').limit(10);
// Phân trang
const page = 2, pageSize = 20;
const rows = await db('orders')
  .orderBy('created_at', 'desc')
  .offset((page - 1) * pageSize)
  .limit(pageSize);
```

❑ Raw SQL

```
const res = await db.raw('select * from ?? order by ?? desc limit ?', ['products', 'price', 10]);

const rows = res.rows;
```

Truy vấn CRUD: Đọc (SELECT)

❑ Sử dụng `.select()` và `.from()` (hoặc `knex('table_name')`).

- ❑ `.select('col1', 'col2')`: Chọn cột cụ thể.
- ❑ `.where('id', 1)`: Thêm điều kiện.
- ❑ `.first()`: Lấy 1 kết quả đầu tiên.
- ❑ `.where({ status: 'active' })`: Điều kiện object.

❑ Knex trả về **Promise**, vì vậy hãy dùng **async/await**.

```
// Lấy tất cả user
const users = await knex
  .select('*')
  .from('users');
```

```
// Lấy user với điều kiện phức tạp
const activeUsers = await knex('users')
  .where({ status: 'active' })
  .andWhere('age', '>', 20);
```

Truy vấn CRUD: Tạo (INSERT)

- ❑ Sử dụng `.insert()`.
- ❑ Phương thức `.returning()` (hỗ trợ bởi Postgres, MSSQL) rất quan trọng để lấy lại dữ liệu vừa chèn (ví dụ: `id` tự tăng).

```
// Chèn một bản ghi
const [newUser] = await knex('users')
  .insert({
    username: 'new_user',
    email: 'new@example.com'
  })
  .returning('*'); // Trả về tất cả cột

// Chèn nhiều bản ghi
await knex('users').insert([
  { username: 'user2', email: 'user2@ex.com' },
  { username: 'user3', email: 'user3@ex.com' }
]);
```

Truy vấn CRUD: Cập nhật & Xóa

- ❑ Cập nhật (UPDATE)
- ❑ Sử dụng `.update()` kết hợp với `.where()`.

```
await knex('users')
  .where('id', 1)
  .update({
    username: 'updated_user',
    last_login: knex.fn.now() // Dùng hàm SQL
  });
```

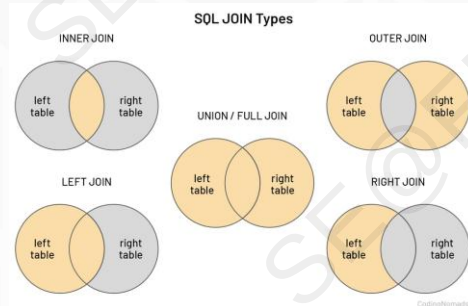
- ❑ Xóa (DELETE)
- ❑ Sử dụng `.del()` (hoặc `.delete()`) kết hợp với `.where()`.

```
// Xóa một user
await knex('users')
  .where('id', 1)
  .del();
```

Kết nối bảng (Joins)

- ❑ **Knex** làm cho việc **join** bảng trở nên rõ ràng và dễ đọc hơn SQL thuần.

- ❑ **join()** (mặc định là innerJoin)
- ❑ **leftJoin()**
- ❑ **rightJoin()**
- ❑ **fullOuterJoin()**



```
const userData = await knex('users')
  .join('profiles', 'users.id', '=', 'profiles.user_id')
  .select(
    'users.username',
    'profiles.avatar'
  )
  .where('users.status', 'active');
```

Giao dịch (Transactions)

- ❑ Đảm bảo tính toàn vẹn dữ liệu (**ACID**). Nếu một truy vấn thất bại, tất cả các truy vấn trong khối **transaction** sẽ được **rollback**.
- ❑ **Knex** cung cấp một **API clean** để xử lý transactions thông qua callback **async/await**.
- ❑ Biến **trx** (transaction object) phải được sử dụng thay cho **knex** cho mọi truy vấn bên trong khối transaction.

```
await trx('order_items').insert({
  order_id: order.id,
  product_id: 99,
  quantity: 1
});
```

CRUD tổng quát

❑ Sử dụng **Repository Pattern**

- ❑ Tạo một **BaseRepository** class.
- ❑ Class này chứa các phương thức CRUD cơ bản (**find**, **findById**, **create**, **update**, **delete**).
- ❑ **Constructor** của class nhận **knex** và **tableName**.

❑ Ưu điểm:

- ❑ Tái sử dụng code, tránh lặp lại (DRY).
- ❑ Trừu tượng hóa lớp truy cập dữ liệu.

❑ Nhược điểm:

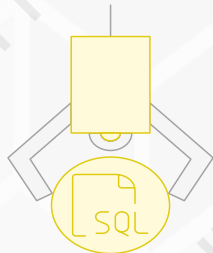
- ❑ **Knex** là **Query Builder**, không phải **ORM**. Sức mạnh của nó là viết **query** cụ thể.
- ❑ Lạm dụng pattern tổng quát có thể làm mất đi lợi ích này và trở nên quá phức tạp khi xử lý logic nghiệp vụ đặc thù (ví dụ: join phức tạp).

Ví dụ

```
class BaseRepository {
  constructor(knex, tableName) {
    this.knex = knex;
    this.tableName = tableName;
  }
  // Lấy tất cả (hoặc lọc theo điều kiện)
  find(where = {}) {
    return this.knex(this.tableName)
      .where(where)
      .select();
  }
  // Lấy theo ID
  findById(id) {
    return this.knex(this.tableName)
      .where('id', id)
      .first();
  }
  // Tạo mới
  create(data) {
    return this.knex(this.tableName)
      .insert(data)
      .returning('*');
  }
}
```

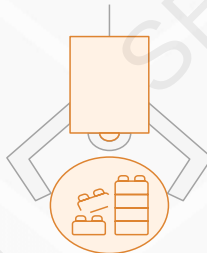
```
// Cập nhật theo ID
update(id, data) {
  return this.knex(this.tableName)
    .where('id', id)
    .update(data);
}
// Xóa theo ID
delete(id) {
  return this.knex(this.tableName)
    .where('id', id)
    .del();
}
```

RAW SQL vs. KNEX vs. ORM



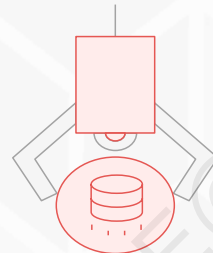
Raw SQL

Linh hoạt tối đa,
nhưng dễ lỗi, khó tái
sử dụng



Knex

Cân bằng; dùng
chain + raw khi cần;
migrations/seeds
tích hợp



ORM (Sequelize/TypeORM/Prisma)

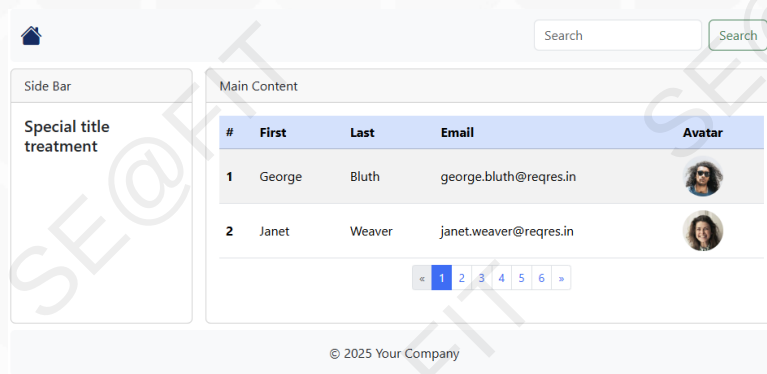
Mô hình hoá entity,
tự map quan hệ;
nhanh cho CRUD
nhưng thực hiện truy
vấn phức tạp khó khăn

Hỏi đáp

Q&A

Bài tập 1

- ❑ Với dữ liệu lấy được từ address: '<https://reqres.in>' cần tạo database trên Supabase. Sau đó xây dựng ứng dụng SSR node.js thực hiện các chức năng migrations, seeds, load dữ liệu và tìm kiếm (có phân trang).



Bài tập 2

- ❑ Chuyển bài tập 1 sang CSR với RESTful API sử dụng: express.js và ReactJS.
 - ❑ Thêm trường password và role
 - ❑ Xây dựng chức năng login, logout, signup, profile và chỉ role "admin" mới xem được danh sách users.